

Effect of Inlet Velocity on Particle Trajectories in LES Channel Flow

Jorge Luis Vargas Lara

December 11, 2025

Abstract

In this short project we use large-eddy simulations (LES) of channel flow in OpenFOAM to explore how inlet velocity affects the transport of sandy particles. Transient velocity fields are post-processed with a Lagrangian particle tracker that accounts for Schiller–Naumann drag, gravity, and wall interactions tailored to the actual channel geometry. We compare particle trajectories, dispersion, and residence times across several inlet velocities and discuss how increased flow rate modifies particle trajectories.

1 Introduction

Turbulent channel flows strongly influence the transport and fate of particulate matter in natural and engineered systems. The degree of turbulence, controlled here via the inlet velocity, can modify particle dispersion, and deposition patterns. There are a wide range of factors involved in the modeling of sandy particles in a natural and/or manmade water system. We want to understand what it takes to implement a very basic model including drag and gravity where particles are non interacting with the use of LES as an added layer of complexity. In this work we ask: *How does changing the inlet velocity in a turbulent channel alter the trajectories and what are the implications for natural and manmade system involving the transport of discrete sandy particles?*

2 Methods

2.1 LES channel flow setup

2.1.1 Geometry and mesh

The simulations are performed using the `pimpleFoam` solver in OpenFOAM with a large-eddy simulation (LES) closure.

The computational domain is a quasi-two-dimensional channel generated with `blockMesh`. All coordinates are specified in millimeters with a global scaling factor of 10^{-3} , so the physical units are meters. The streamwise coordinate x ranges from $x_{\min} = -20.6$ mm to $x_{\max} = 290$ mm, giving a total length

$$L_x \approx 0.311 \text{ m.}$$

The channel height varies from an inlet height

$$H_{\text{in}} = 25.4 \text{ mm}$$

up to a maximum height of $H_{\max} = 50.8$ mm in the main section. The spanwise direction z has a constant thickness of 1 mm, and an `empty` boundary condition is applied, so the configuration is effectively two-dimensional.

The mesh is a structured hexahedral grid composed of five blocks with local resolutions

- Inlet/contraction block: $18 \times 30 \times 1$ cells with mild streamwise grading,
- Two central channel blocks: $180 \times 27 \times 1$ and $180 \times 30 \times 1$ cells,
- Two outlet/diffuser blocks: $25 \times 27 \times 1$ and $25 \times 30 \times 1$ cells with streamwise grading.

The total cell count is $N_{\text{cells}} = 12\,225$, corresponding to 428 cells in the streamwise direction and roughly 27–30 cells across the height of the channel.

2.1.2 Boundary and initial conditions

At the inlet plane we impose a uniform streamwise velocity

$$\mathbf{u}_{\text{in}} = (U_{\text{in}}, 0, 0),$$

with U_{in} varied between cases in the range

$$U_{\text{min}} = 0.03 \text{ m/s}, \quad U_{\text{max}} = 4.0 \text{ m/s},$$

so that

$$U_{\text{min}} \leq U_{\text{in}} \leq U_{\text{max}}.$$

The boundary conditions for velocity and pressure are:

- **Inlet:** `fixedValue` for $\mathbf{u}$ with the specified U_{in} ; `zeroGradient` for kinematic pressure p .
- **Outlet:** `zeroGradient` for $\mathbf{u}$; `fixedValue` for p (set to zero gauge pressure).
- **Walls (upper and lower):** no-slip (`noSlip`) for $\mathbf{u}$; `zeroGradient` for p . The subgrid viscosity ν_t uses a standard `nutkWallFunction`.
- **Front/back:** `empty` in the z direction, yielding a 2D LES.

The initial condition for velocity is $\mathbf{u}(\mathbf{x}, 0) = \mathbf{0}$, and the initial pressure field is set to $p(\mathbf{x}, 0) = 0$.

2.1.3 Fluid properties and turbulence model

The working fluid is treated as Newtonian with constant kinematic viscosity

$$\nu = 1.0 \times 10^{-5} \text{ m}^2/\text{s}.$$

LES turbulence modelling is enabled via the `Smagorinsky` model with cube-root-of-volume filter width (`delta = cubeRootVol`). The `turbulence` flag is set to `on`, and model coefficients are printed for verification.

2.1.4 Temporal discretization and solvers

The unsteady flow is integrated in time with a first-order implicit Euler scheme (`ddtSchemes{default Euler}`). The numerical schemes are:

- Gradients: `Gauss linear`,
- Divergence: `Gauss linearUpwind grad(U)` for $\nabla \cdot (\phi \mathbf{U})$ and `Gauss linear` for the viscous stress term,
- Laplacians: `Gauss linear corrected`,
- Surface interpolation: `linear`,

- Normal gradients: `corrected`.

The coupled pressure–velocity system is solved with the PIMPLE algorithm using

- `GAMG` for pressure (p and $pFinal$) with `DICGaussSeidel` smoothing,
- `smoothSolver` for velocity (U and $UFinal$) with `symGaussSeidel` smoothing.

Standard tolerances are employed: 10^{-7} for pressure (with small relative tolerance during intermediate iterations) and 10^{-5} for velocity.

2.1.5 Time step control and sampling

The simulation is run from $t = 0$ to $t = 3$ s with an initial time step $\Delta t = 10^{-4}$ s. Adaptive time stepping is activated with a maximum Courant number $Co_{\max} = 5$. Field data are written using `writeControl = adjustableRunTime` every $\Delta t_{\text{write}} = 0.01$ s. These written time directories are later used as the basis for particle tracking.

Figure 1: Suggested Plot 1: Mean streamwise velocity profiles or representative snapshots of the streamwise velocity field for different inlet velocities. This figure documents the flow regimes obtained in each case.

2.2 Flow field post-processing

The transient velocity field $\mathbf{u}(\mathbf{x}, t)$ is sampled from all OpenFOAM time directories after the initial condition ($t > 0$). For each saved time level t_n we read the cell-centred velocity

$$\mathbf{u}(\mathbf{x}_i, t_n), \quad i = 1, \dots, N_{\text{cells}},$$

and stack these into a three-dimensional array $U_{ni\alpha}$ with indices for time n , cell i , and velocity component $\alpha \in \{x, y, z\}$. Cell centres $\mathbf{x}_i$ are obtained via the OpenFOAM `C` field and accessed using a Python interface (`foamlib`). A KD-tree built from $\{\mathbf{x}_i\}$ is used for efficient nearest-neighbour searches when interpolating $\mathbf{u}$ to particle positions. For intermediate particle times lying between two saved CFD times t_n and t_{n+1} , the fluid velocity is obtained by linear interpolation in time between these fields.

2.3 Lagrangian particle tracking

2.3.1 Particle model and properties

Particles are treated as non-interacting point particles with diameter d_p and density ρ_p , transported by the local fluid velocity and subject to drag and gravity. The particle position $\mathbf{x}_p$ and velocity $\mathbf{u}_p$ evolve according to

$$m_p \frac{d\mathbf{u}_p}{dt} = \mathbf{F}_D + \mathbf{F}_g, \tag{1}$$

$$\frac{d\mathbf{x}_p}{dt} = \mathbf{u}_p, \tag{2}$$

where m_p is the particle mass, $\mathbf{F}_g$ is the net gravitational (weight minus buoyancy) force, and $\mathbf{F}_D$ is the drag force.

The particle parameters used here correspond to very fine, wet sand:

$$d_p = 6.25 \times 10^{-5} \text{ m}, \quad \rho_p = 1840 \text{ kg/m}^3,$$

with fluid density and dynamic viscosity

$$\rho_f = 1000 \text{ kg/m}^3, \quad \mu = 1.0 \times 10^{-3} \text{ Pa s.}$$

Gravity acts in the negative y -direction,

$$\mathbf{g} = (0, -9.81, 0) \text{ m/s}^2,$$

so particles experience gravitational settling across the channel height. The particle volume V_p , mass m_p , and projected area A_p are

$$V_p = \frac{\pi}{6} d_p^3, \quad m_p = \rho_p V_p, \quad A_p = \frac{\pi}{4} d_p^2.$$

The gravitational (weight minus buoyancy) force is

$$\mathbf{F}_g = (\rho_p - \rho_f) V_p \mathbf{g},$$

which is constant in time.

2.3.2 Schiller–Naumann drag and interpolation

The drag force is modelled using the Schiller–Naumann correlation. Given the fluid velocity at the particle position, $\mathbf{u}_f = \mathbf{u}(\mathbf{x}_p, t)$, we define the slip velocity $\mathbf{u}_{\text{rel}} = \mathbf{u}_f - \mathbf{u}_p$ with magnitude $|\mathbf{u}_{\text{rel}}|$. The particle Reynolds number is

$$\text{Re}_p = \frac{\rho_f d_p |\mathbf{u}_{\text{rel}}|}{\mu},$$

capped at $\text{Re}_p \leq 10^5$ to avoid unphysical values. The drag coefficient is

$$C_D = \begin{cases} \frac{24}{\text{Re}_p} (1 + 0.15 \text{Re}_p^{0.687}), & \text{Re}_p < 1000, \\ 0.44, & \text{Re}_p \geq 1000, \end{cases}$$

and the resulting drag force is

$$\mathbf{F}_D = \frac{1}{2} \rho_f C_D A_p |\mathbf{u}_{\text{rel}}| \mathbf{u}_{\text{rel}}.$$

The fluid velocity at a particle position and time, $\mathbf{u}(\mathbf{x}_p, t)$, is obtained by:

- **Nearest-neighbour interpolation in space:** a `ckdTree` built from the cell centres $\{\mathbf{x}_i\}$ is used to find the closest cell to $\mathbf{x}_p$.
- **Linear interpolation in time:** if t lies between two saved CFD times t_n and t_{n+1} , the velocity is interpolated as

$$\mathbf{u}(\mathbf{x}_p, t) \approx (1 - \alpha) \mathbf{u}(\mathbf{x}_p, t_n) + \alpha \mathbf{u}(\mathbf{x}_p, t_{n+1}),$$

with $\alpha = (t - t_n)/(t_{n+1} - t_n)$.

In the implementation, particle speeds are also capped at ten times the maximum magnitude of the fluid velocity observed in the LES fields to prevent numerical blow-up.

2.3.3 Seeding, boundaries, and integration

Particle trajectories are integrated by a Python function `run_particles(case, ...)` that operates on a completed `FoamCase`. We use a single burst of particles per simulation. In the reference LES case we typically track on the order of $n_p \sim \mathcal{O}(10^2)$ particles.

Seeding is tailored to the actual `blockMesh` geometry. Let $x_{\text{in,L}}$ denote the left x -coordinate of the inlet box (here $x_{\text{in,L}} = -20.6$ mm) and x_{con} the end of the contraction ($x_{\text{con}} = 0$). Particles are placed slightly inside the inlet block at

$$x_p(t_0) = x_{\text{seed}} = x_{\text{in,L}} + 0.2(x_{\text{con}} - x_{\text{in,L}}),$$

so that no particles start outside the physical inlet. The vertical coordinates are sampled uniformly between the local bottom and top walls at x_{seed} ,

$$y_p(t_0) \in [y_{\text{bot}}(x_{\text{seed}}), y_{\text{top}}(x_{\text{seed}})],$$

where $y_{\text{bot}}(x)$ and $y_{\text{top}}(x)$ are piecewise-linear wall profiles that reproduce the inlet box, main rectangular channel, and tapered outlet sections specified in `blockMeshDict`. The spanwise coordinate is fixed at the mid-plane,

$$z_p(t_0) = \frac{1}{2}(z_{\text{min}} + z_{\text{max}}),$$

and initial particle velocities are set to zero, $\mathbf{u}_p(t_0) = \mathbf{0}$.

The CFD data provide a set of saved times $\{t_n\}$ corresponding to the written OpenFOAM time directories. Let

$$\Delta t_{\text{CFD}}^{\text{min}} = \min_n (t_{n+1} - t_n)$$

denote the smallest interval between written fields. The particle time step is chosen as

$$\Delta t_p = \frac{\Delta t_{\text{CFD}}^{\text{min}}}{100},$$

so that each CFD interval is resolved by approximately 100 particle substeps. This ensures that particles “see” every saved velocity field and that the drag dynamics remain stable.

Particle positions and velocities are advanced with an explicit Euler scheme,

$$\mathbf{u}_p^{k+1} = \mathbf{u}_p^k + \frac{\mathbf{F}_D^k + \mathbf{F}_g}{m_p} \Delta t_p, \quad (3)$$

$$\mathbf{x}_p^{k+1} = \mathbf{x}_p^k + \mathbf{u}_p^{k+1} \Delta t_p. \quad (4)$$

At each substep the fluid velocity $\mathbf{u}_f$ at $\mathbf{x}_p^k$ is obtained by nearest-neighbour interpolation in space (KD-tree on cell centres) and linear interpolation in time between the two bracketing CFD times. To prevent unphysical numerical blow-up, particle speeds are capped at

$$|\mathbf{u}_p| \leq 10 \max_{n,i} |\mathbf{u}(\mathbf{x}_i, t_n)|,$$

and any particle that develops non-finite coordinates (NaN or $\pm\infty$) is marked inactive and removed from further statistics.

Boundary interactions are implemented directly in physical space, consistent with the `blockMesh` geometry:

- **Outlet:** once a particle crosses the downstream plane at the end of the domain (maximum x of the cell centres) it is considered to have exited and is marked inactive.
- **Inlet left wall:** if a particle attempts to move past the inlet left wall ($x < x_{\text{in,L}}$), it is reflected back to $x = x_{\text{in,L}}$ with its streamwise velocity component reversed if it points outward.

- **Top and bottom walls:** at any streamwise location x , the vertical bounds are enforced using the functions $y_{\text{bot}}(x)$ and $y_{\text{top}}(x)$ that encode the inlet, main channel, and tapered outlet heights. If y_p crosses one of these walls, it is clamped back onto the wall and the corresponding velocity component is reflected.
- **Spanwise walls:** the z -coordinate is restricted to the range $[z_{\text{min}}, z_{\text{max}}]$ defined by the cell centres; if a particle crosses this range the position is clamped and the z -velocity component is reflected.

Trajectories are saved at every written CFD time, yielding a three-dimensional array of particle positions of shape $(N_{\text{save}}, n_p, 3)$, where N_{save} is the number of OpenFOAM time directories with velocity fields. These data are used both for post-processing statistics and for visualisation of particle paths (e.g. in ParaView or Python).

3 Results

This section should describe and compare the results of the pressure and velocity field. This shows a transient behavior where eddies and other turbulent structures are moving in the velocity images given the higher and lower region seen in this paper. For figure 3 regions of low pressure are not stationary given we increase the velocity at the inlet. This trend continues for each given experiment. This is important to keep in mind these transient changes in the pressure and velocity field will be impacting the trajectory of injected particles overtime.

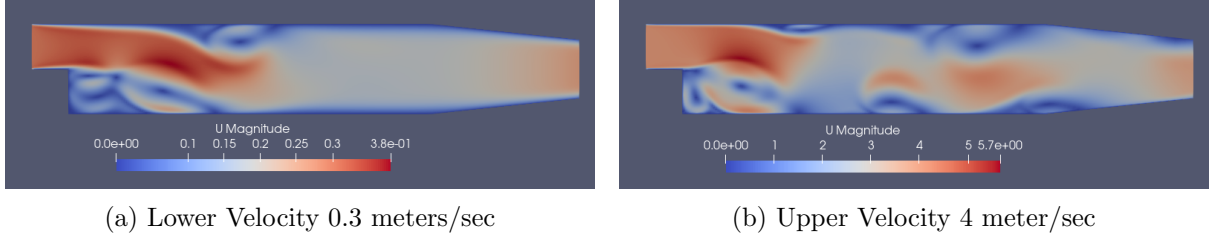
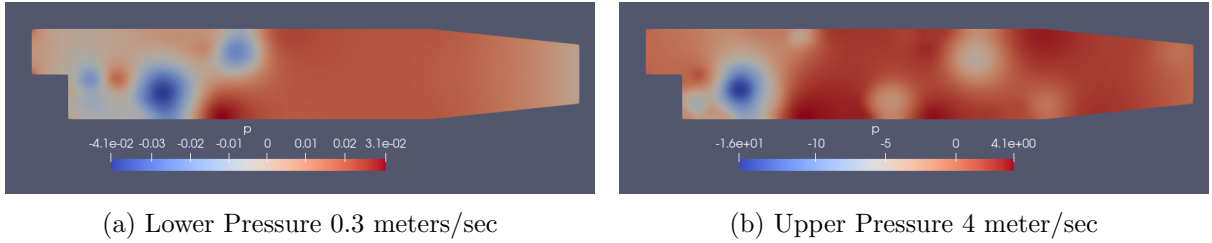


Figure 2: Lower and Upper Velocity behavior, image at 1 second of simulation time.



Given that the velocity and pressure fields evolve in time for each inlet-velocity case, we first tested a range of particle parameters before running the full set of simulations. In Figure 4 we show an example with a relatively large number of particles ($N_p = 200$) and a comparatively large particle diameter, $d_p = 2.0 \times 10^{-3}$ m. Among the parameters explored, the most sensitive one was the particle density. For these heavy particles, the trajectories show an almost complete neglect of the turbulent structures: the eddies are not strong enough, at any inlet-velocity condition considered here, to move particles away from their nearly arch-like, laminar-looking streamlines. At face value, these trajectories indicate that gravitational settling is the dominant process in this regime.

For the remainder of this paper, we therefore use a reduced particle density of $\rho_p = 1840$ kg/m³ and a smaller particle diameter of $d_p = 6.25 \times 10^{-5}$ m. Particle count was also reduced from $N_p = 200$ to $N_p = 10$ to better visualize the individual behavior of particles.

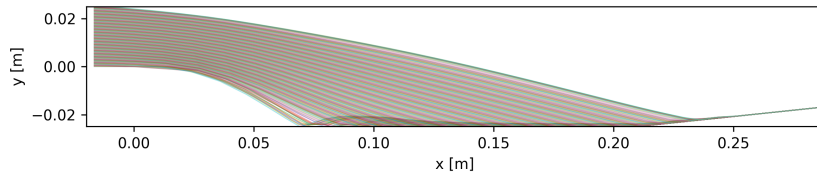
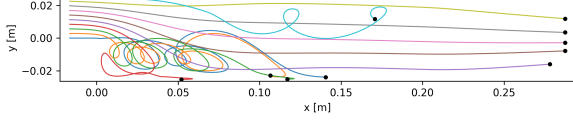
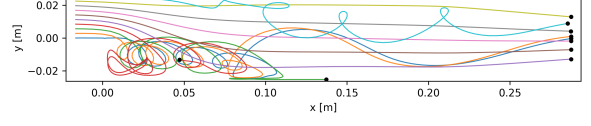


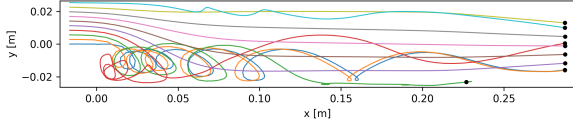
Figure 4: Particle trajectories for an inlet velocity of 1.5 m/s. Number of particles: $N_p = 200$; particle density $\rho_p = 2650$ kg/m³; particle diameter $d_p = 2.0 \times 10^{-3}$ m.



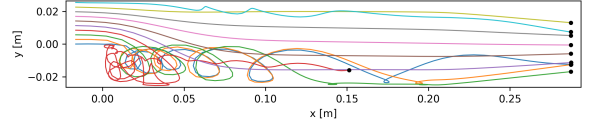
(a) $U_{\text{in}} = 0.3 \text{ m/s}$



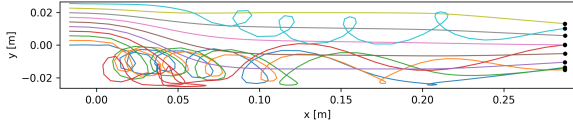
(b) $U_{\text{in}} = 0.5 \text{ m/s}$



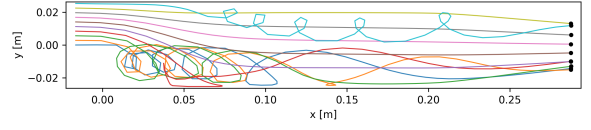
(c) $U_{\text{in}} = 0.7 \text{ m/s}$



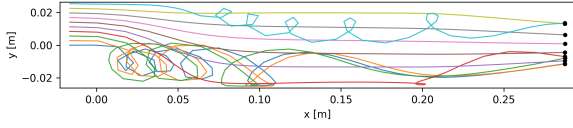
(d) $U_{\text{in}} = 0.9 \text{ m/s}$



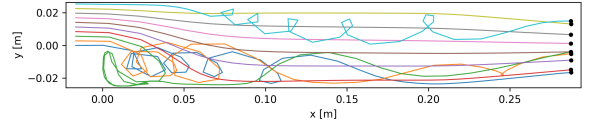
(e) $U_{\text{in}} = 1.2 \text{ m/s}$



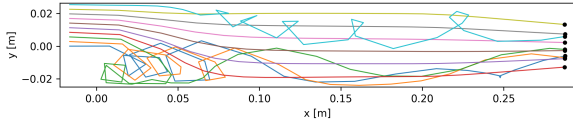
(f) $U_{\text{in}} = 1.5 \text{ m/s}$



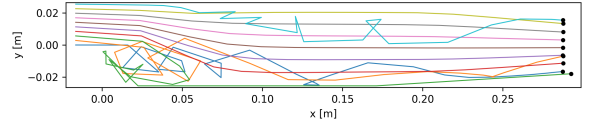
(g) $U_{\text{in}} = 1.7 \text{ m/s}$



(h) $U_{\text{in}} = 2.0 \text{ m/s}$



(i) $U_{\text{in}} = 3.0 \text{ m/s}$



(j) $U_{\text{in}} = 4.0 \text{ m/s}$

Figure 5: Particle trajectories projected onto the x - y plane for different inlet velocities U_{in} . Each panel shows Lagrangian paths of very fine, dense particles transported under gravity and Schiller–Naumann drag in the LES channel.

4 Discussion and conclusions

We have demonstrated that varying the inlet velocity in an LES channel flow has a clear impact on particle trajectories and residence times for very fine, dense particles transported under gravity and Schiller–Naumann drag. This model, given its simplicity of including only drag and gravitational forces without particle–particle interactions, demonstrates three fundamental principles of Lagrangian particle tracking.

First, turbulent structures are most likely to occur where the geometry of the solid surface undergoes a sharp change in angle. At the beginning of our domain, the width of the flow path narrows abruptly, providing an opportunity for recirculating flow to occur and turbulent structures to be generated near the bottom of the domain. These turbulent structures move through the fluid domain, rotating and dissipating energy as they travel downstream.

Second, particles injected into the flow domain are inclined to follow an arch-shaped path when subject primarily to gravity. Fluid drag tends to flatten this gravitational arch due to the resisting effect induced by the viscosity of the fluid medium (in this case, water). As demonstrated in Figure 4, the heavy particle trajectories indeed display this arch-like behavior. Although turbulence is present in the simulation, its influence on these large, dense particles is effectively filtered out because of the high particle-to-fluid density ratio associated with $d_p = 2.0 \times 10^{-3}$ m.

Third, by reducing the particle density, we finally observe the effects of the turbulent structures dominating the particle trajectories. In all cases, the recirculating region produces coherent turbulent structures that temporarily trap particles, causing them to remain within the eddy for several loops before they are eventually released. Ultimately, the majority of particles escape the recirculating region and continue downstream, although their motion is continually slowed or redirected by the turbulent structures propagating through the fluid domain.

In sand-like particles, turbulence can dominate their motion only when the particle density is not substantially different from that of the surrounding fluid. The major factor that dictates how much deviation we can expect from the standard arch-shaped trajectory—produced solely by drag and gravity—depends strongly on the surface geometry where the flow occurs. This conclusion is supported by the fact that the only constant across all our simulations, without deeper technical analysis, is the sharp change in channel width near the inlet. In that region we consistently observe a recirculating zone that generates turbulent structures which not only capture particles and eventually release them, but also send turbulent motions downstream, further disrupting the standard paths that particles would otherwise follow.

5 References

@manualOpenFOAM-2406, title = OpenFOAM v2406 User Guide, author = OpenFOAM Foundation, year = 2024, note = Available at <https://www.openfoam.com/releases/openfoam-v2406>,

@miscfoamlib-v1, title = foamlib: Python Tools for Interacting with OpenFOAM Cases, author = Henderson, Harry and contributors, year = 2024, howpublished = <https://github.com/hhp2/foamlib>, note = Version used: commit `insert-hash`; accessed 2025-02-19